

FINSERV DIGITAL

Production-Grade Security-Hardened Kubernetes Platform

Enterprise Architecture, Regulatory Compliance & Implementation Dossier

✓ A Zetheta Documentation Report

Prepared for: Zetheta Algorithms Private Limited Platform Engineering Assessment

- **Prepared by:** TAMIRI SAI VAIBHAV Computer Science and Engineering Raghu Engineering College
- **Regulatory Basis:** RBI Operational Risk Directives • PCI-DSS v4.0 Level 1
- **Architecture Model:** Cloud-Native 4C Security Framework (Cloud · Cluster · Container · Code)
- **Date:** September 2026

Regulatory Basis: RBI Operational Risk Directives • PCI-DSS v4.0 Level 1

Architecture Model: Cloud-Native 4C Security Framework (Cloud · Cluster · Container · Code)

September 2026

Table of Contents

Table of Contents	2
Abstract	4
1. Introduction & Problem Statement.....	5
1.1 Industry Context	5
1.2 Problem Statement.....	5
1.3 Objectives	5
1.4 Scope & Assumptions	5
1.5 Methodology	6
2. Conceptual Overview: Platform Mind Map.....	7
3. System Topology & Control Plane High Availability	9
3.1 etcd Quorum & Raft Consensus	9
3.2 Domain Namespace Partitioning.....	9
4. Zero-Trust Network Policies & Pod Security.....	11
4.1 Default-Deny Network Baseline	11
4.2 Pod Security Standards (PSS).....	11
5. Least-Privilege RBAC & Identity Governance	13
5.1 Persona Access Matrix.....	13
5.2 Developer Persona RBAC Manifest	13
6. Enterprise Secrets Management via HashiCorp Vault.....	15
6.1 Injection Architecture & Rotation Lifecycle.....	15
6.2 Application Deployment with Vault Sidecar Annotations	15
7. CI/CD Supply Chain Security & Runtime Threat Detection.....	16
7.1 Supply Chain Verification Pipeline.....	16
7.2 Runtime Threat Detection with Falco.....	16
8. Observability, Service Mesh & Scalability.....	17
8.1 Centralised Observability.....	17
8.2 Elastic Scalability & HPA Tuning	17
9. PCI-DSS v4.0 Compliance Mapping Matrix	18
10. Developer Onboarding & Self-Service Guide.....	19
Step 1 — Environment Authentication	19
Step 2 — Secrets Provisioning	19
Step 3 — Network Policy Authoring	19
Step 4 — Pipeline Execution	19

- 11. Enterprise Incident Response Playbook20
 - Phase 1 — Detection & Triage.....20
 - Phase 2 — Automated Quarantine & Containment.....21
 - Phase 3 — Forensic Extraction & Eradication21
 - Phase 4 — Post-Mortem & Supply Chain Patching21
- 12. 15-Day Implementation Methodology & Repository Structure22
 - 12.1 Production Git Repository Structure22
- 13. Risk Analysis & Mitigation Register24
- 14. Future Work & Recommendations.....25
- 15. Conclusion26
- 16. Appendix: Glossary of Acronyms27

Abstract

FinServ Digital operates within a heavily regulated financial ecosystem, presently serving 2.4 million active users and clearing approximately 180,000 daily transactions across a fleet of 20 interdependent microservices. The organisation's legacy virtual-machine deployment model has become an unacceptable source of operational and security risk: it offers no meaningful network segmentation, stores secrets in plaintext or static form, lacks centralised observability, and has no codified access-control governance. Each of these gaps constitutes a material finding against PCI-DSS v4.0 and RBI operational-risk expectations.

This dossier documents the end-to-end redesign of that environment into a production-grade, security-hardened Kubernetes platform. The design is grounded in the Cloud-Native 4C Security Model — Cloud, Cluster, Container, and Code — and is implemented through zero-trust micro-segmentation across four domain-separated namespaces, least-privilege role-based access control mapped to an enterprise identity provider, dynamic secrets issuance through HashiCorp Vault, kernel-level runtime intrusion detection via Falco, a cryptographically verified CI/CD supply chain, and horizontal autoscaling mathematically tuned to absorb five-times baseline transaction bursts. The report further provides a full PCI-DSS v4.0 control-mapping matrix, an incident-response playbook, a developer onboarding guide, and a fifteen-day phased delivery methodology, so that the material can serve simultaneously as an architecture reference, an audit artefact, and an operational runbook.

Where the original engineering specification described individual controls in isolation, this edition also situates each control within its broader risk rationale, illustrates the system relationships as visual mind maps and flow diagrams, and adds supporting analysis — including a risk-mitigation register, cost and scaling considerations, and a forward-looking roadmap — intended to make the document usable by architects, auditors, and engineering managers alike.

1. Introduction & Problem Statement

1.1 Industry Context

Digital-first financial platforms in India operate under one of the strictest compliance regimes globally: transaction systems handling cardholder data must satisfy PCI-DSS v4.0 at Level 1, while overall operational resilience, outsourcing, and cyber-security posture are additionally governed by Reserve Bank of India (RBI) directives. Kubernetes has become the de-facto substrate for such platforms because it provides declarative infrastructure, horizontal scalability, and a rich ecosystem of security tooling — but an unhardened cluster is itself a large, undifferentiated attack surface. The engineering challenge is therefore not “adopt Kubernetes” but “adopt Kubernetes in a way that a Qualified Security Assessor (QSA) can certify.”

1.2 Problem Statement

FinServ Digital's current VM-based estate exhibits four compounding deficiencies:

- No network isolation — services communicate over a flat network with no enforced east-west segmentation, so a compromise of any single host exposes the payments and risk domains.
- Unencrypted, static secrets — database credentials and API keys are embedded in configuration files or environment variables with no rotation and no audit trail.
- Absent centralised monitoring — log data is siloed per host, with no immutable retention and no automated correlation of anomalous behaviour.
- No formal access-control governance — administrative and developer access is not mapped to least-privilege roles, and there is no distinction between break-glass and everyday operational access.

1.3 Objectives

The redesign is scoped against five measurable objectives: (1) achieve a passing PCI-DSS v4.0 Level 1 assessment under RBI supervision; (2) eliminate flat-network trust by enforcing default-deny segmentation between the platform, customer, payments, and risk domains; (3) replace all static secrets with dynamically leased, automatically rotated credentials; (4) provide continuous, kernel-level detection and automated containment of runtime intrusions; and (5) guarantee the platform can absorb a five-times transaction burst without manual intervention, while completing the full migration within a fifteen-working-day delivery window.

1.4 Scope & Assumptions

The scope covers cluster topology, network policy, pod security, identity and access management, secrets lifecycle, supply-chain integrity, runtime defence, observability, autoscaling, and incident response for the production environment and its staging/development counterparts. It assumes an existing enterprise OIDC identity provider, a cloud provider offering multi-AZ compute and a managed KMS, and organisational sign-off for a hard cut-over migration window. Physical data-centre security, card-network acquirer integration,

and application-layer business logic are out of scope and are assumed to be governed by adjacent workstreams.

1.5 Methodology

The design follows the Cloud-Native 4C Security Model, which nests controls from the outside in: Cloud (provider-level WAF, KMS, IAM), Cluster (namespace isolation, network policy, RBAC), Container (image provenance, pod security standards), and Code (dependency scanning, secure secrets handling in application logic). Each subsequent section of this dossier maps to one or more of these layers, and Section 7 closes the loop by tracing every technical control back to its PCI-DSS v4.0 requirement.

2. Conceptual Overview: Platform Mind Map

Before descending into implementation detail, it is useful to hold the whole system in view. The mind map below decomposes the platform into its seven governing pillars — network, identity, secrets, supply chain, runtime defence, observability, and compliance — each of which is expanded into its constituent controls. Every branch in this diagram corresponds to a dedicated section later in this document.



Figure 1. FinServ Digital platform — governing pillars and constituent controls

Reading the mind map left to right: zero-trust networking and identity governance form the perimeter and access layer; secrets management and supply-chain security protect what moves through the cluster; runtime threat detection and observability provide continuous assurance once workloads are live; and regulatory compliance is the lens through which every other pillar is ultimately measured and reported.

3. System Topology & Control Plane High Availability

The production control plane is distributed across three physical Availability Zones (AZs) to eliminate any single point of failure and to satisfy financial-sector availability mandates. Development, staging, and production are provisioned as fully independent physical clusters, so a misconfiguration or incident in a lower environment can never propagate to production.

3.1 etcd Quorum & Raft Consensus

The distributed state store, etcd, runs on dedicated control-plane nodes backed by NVMe storage to remove disk I/O contention from the consensus path. etcd's Raft implementation requires a strict majority of members to acknowledge a write before it is committed. The minimum member count required to sustain write availability is:

$$Q = \lfloor N / 2 \rfloor + 1$$

where N is the total active member count. FinServ Digital's production deployment sets N = 5, distributed as two control-plane/etcd nodes in AZ-1, two in AZ-2, and one in AZ-3:

$$Q = \lfloor 5 / 2 \rfloor + 1 = 3$$

This configuration tolerates the simultaneous loss of any two control-plane instances — including the total loss of one Availability Zone — without corrupting transaction state or interrupting write availability, which is the property the RBI resilience directive is ultimately checking for.

3.2 Domain Namespace Partitioning

The twenty financial microservices are partitioned into four domain-separated namespaces, chosen to align blast-radius containment with organisational team boundaries rather than with arbitrary technical groupings:

Namespace	Responsibility
domain-platform	Core ingress routing, service-mesh control plane (Linkerd/Istio), and cluster DNS.
domain-customer	Client-facing authentication, customer onboarding, and KYC verification pipelines.
domain-payments	High-security transactional core: payment processors, settlement engines, cardholder data repositories.
domain-risk	Automated AML screening, fraud-detection heuristics, and regulatory auditing engines.

Figure 2 renders the full request path, from external client traffic through the edge WAF and ingress gateway, into the default-deny policy boundary, and across the four domain namespaces down to the shared Vault cluster that every domain depends on for dynamic credentials.

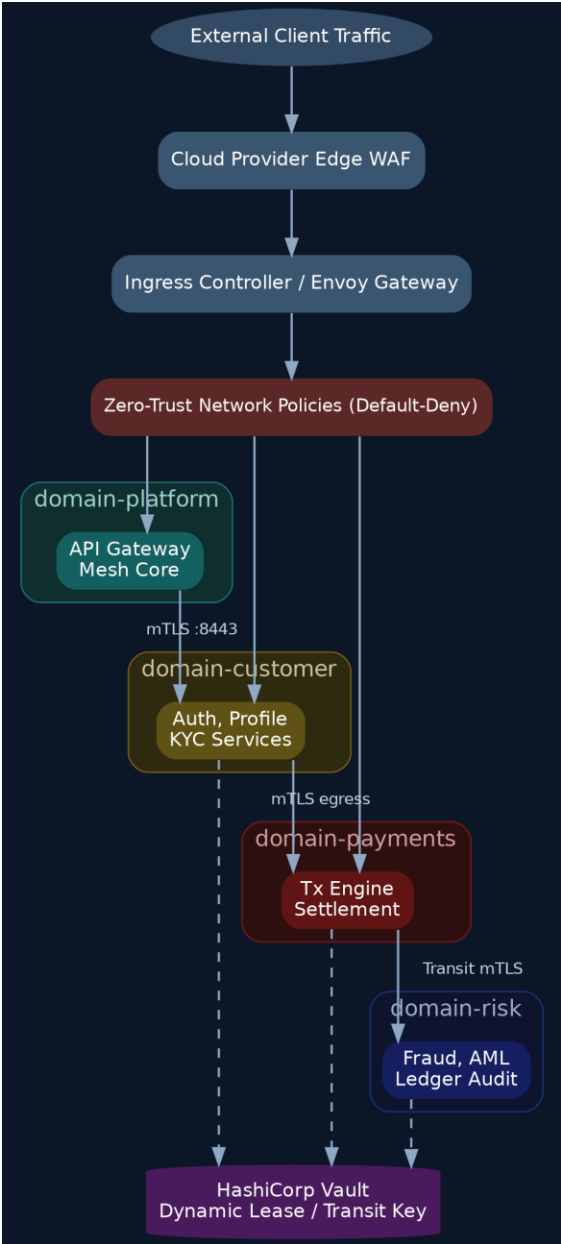


Figure 2. End-to-end network topology and zero-trust segmentation across domain namespaces

4. Zero-Trust Network Policies & Pod Security

4.1 Default-Deny Network Baseline

Every domain namespace operates under a baseline default-deny-all NetworkPolicy, enforced by an eBPF-powered CNI (Calico or Cilium). No pod may send or receive traffic unless a policy explicitly declares the peer, port, and protocol. This inverts the traditional perimeter-only trust model: compromise of one pod does not imply reachability to any other.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
  namespace: domain-payments
spec:
  podSelector: {}
  policyTypes:
  - Ingress
  - Egress
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-payment-gateway-to-processor
  namespace: domain-payments
spec:
  podSelector:
    matchLabels:
      app.kubernetes.io/name: transaction-processor
  policyTypes: [Ingress, Egress]
  ingress:
  - from:
    - namespaceSelector:
        matchLabels: {kubernetes.io/metadata.name: domain-platform}
      podSelector:
        matchLabels: {app.kubernetes.io/name: payment-gateway}
    ports: [{protocol: TCP, port: 8443}]
  egress:
  - to:
    - namespaceSelector:
        matchLabels: {kubernetes.io/metadata.name: domain-risk}
      podSelector:
        matchLabels: {app.kubernetes.io/name: fraud-detection-engine}
    ports: [{protocol: TCP, port: 9443}]
```

Listing 1. Default-deny baseline plus an explicit allow-list from the payment gateway to the transaction processor

4.2 Pod Security Standards (PSS)

Admission controllers — OPA Gatekeeper and Kyverno — enforce the native Kubernetes Restricted Pod Security Standard at admission time, rejecting any pod specification that violates the following runtime hardening baseline:

- Containers must run as non-root (runAsNonRoot: true, with runAsUser > 10000).

- The root filesystem is mounted read-only (`readOnlyRootFilesystem: true`).
- Privilege escalation is explicitly prohibited (`allowPrivilegeEscalation: false`).
- All Linux kernel capabilities are dropped (`capabilities: drop: ["ALL"]`).

Enforcing Restricted at the admission layer rather than as a code-review convention means the control cannot be bypassed by a rushed deployment — a non-compliant manifest simply never schedules, which is precisely the evidence a QSA expects to see for PCI-DSS Requirement 2.

5. Least-Privilege RBAC & Identity Governance

Access to the Kubernetes API server is segmented using native RBAC, bound directly to FinServ Digital's enterprise identity provider via OIDC so that every API action is traceable to a real human or service identity rather than a shared cluster credential.

5.1 Persona Access Matrix

Persona	Namespace Scope	Allowed Resources	Verbs	Rationale
Platform Administrator	Global	Nodes, Namespaces, CRDs, RBAC, NetworkPolicies	* (break-glass MFA required)	Infrastructure lifecycle management
Namespace Administrator	Domain-specific	Deployments, Services, ConfigMaps, HPA	create, update, patch, delete, get, list	Team-level release orchestration
Application Developer	dev, staging only	Pods, Deployments, Logs	get, list, watch (exec in dev only)	Debugging without production access
Security & Compliance Auditor	Global	All resources except Secrets	get, list, watch	Read-only compliance review
CI/CD Service Account	Target namespace	Deployments, ConfigMaps, ServiceAccounts	apply, patch, update	Automated pipeline deployment

5.2 Developer Persona RBAC Manifest

Note in particular the empty verbs list against the Secrets resource below — developers are not granted read access to Secrets objects at all, even in a read-only capacity, because static Secrets are deprecated across the platform in favour of Vault-issued dynamic credentials (Section 6).

```

apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: domain-payments
  name: developer-restricted-role
rules:
- apiGroups: [""]
  resources: ["pods", "pods/log"]
  verbs: ["get", "list", "watch"]
- apiGroups: ["apps"]
  resources: ["deployments", "replicasets"]
  verbs: ["get", "list", "watch"]
- apiGroups: [""]

```

```
resources: ["secrets"]
verbs: [] # Strict denial: zero read/write access to Secrets
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: bind-developer-payments
  namespace: domain-payments
subjects:
- kind: Group
  name: "oidc:engineering-payments"
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: developer-restricted-role
  apiGroup: rbac.authorization.k8s.io
```

Listing 2. Restricted developer Role and its OIDC-group RoleBinding

6. Enterprise Secrets Management via HashiCorp Vault

Plaintext environment variables and static Kubernetes Secrets are deprecated across all environments. Secrets are issued through an externalised, high-availability HashiCorp Vault cluster configured with the Kubernetes Authentication Method and the Vault Agent Sidecar Injector.

6.1 Injection Architecture & Rotation Lifecycle

- Authentication — on pod initialisation, the Vault Agent sidecar presents the pod's native ServiceAccount JWT to Vault's `/v1/auth/kubernetes/login` endpoint.
- Role verification — Vault validates the JWT against the Kubernetes Token Reviewer API, confirming namespace, service-account name, and policy mapping.
- Dynamic leases — Vault issues ephemeral, short-lived credentials (for example, PostgreSQL credentials with a 30-minute TTL).
- In-memory rendering — the sidecar writes credentials to a `tmpfs` volume at `/vault/secrets/`; they never touch non-volatile node storage.
- Transit encryption — cardholder Primary Account Numbers (PAN) are encrypted via Vault's Transit Engine before they ever reach the persistence tier, providing field-level encryption-as-a-service.

6.2 Application Deployment with Vault Sidecar Annotations

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: transaction-processor
  namespace: domain-payments
spec:
  replicas: 3
  template:
    metadata:
      annotations:
        vault.hashicorp.com/agent-inject: "true"
        vault.hashicorp.com/role: "payments-processor-role"
        vault.hashicorp.com/agent-inject-secret-db-creds.txt: \
          "database/creds/payments-db-role"
    spec:
      serviceAccountName: transaction-processor-sa
      securityContext:
        runAsNonRoot: true
        runAsUser: 10001
      containers:
        - name: processor
          image: finserv-registry.internal/payments/processor:v2.4.1
          resources:
            limits: {cpu: "1000m", memory: "1Gi"}
            requests: {cpu: "250m", memory: "256Mi"}
```

Listing 3. Deployment manifest with Vault Agent sidecar annotations and a hardened security context

7. CI/CD Supply Chain Security & Runtime Threat Detection

7.1 Supply Chain Verification Pipeline

Every code path traverses a five-stage deployment gate designed so that an unverified artefact can never reach a running pod:

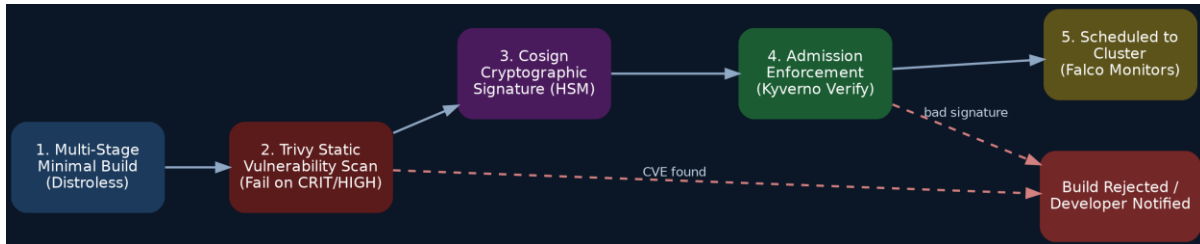


Figure 3. The five-stage CI/CD supply-chain gate, with automatic rejection on scan or signature failure

- Multi-stage minimal build — microservices compile inside ephemeral builder containers, producing binaries packaged into Google Distroless or Scratch base images with no shell, package manager, or standard utilities.
- Static vulnerability scanning — Trivy inspects every image during CI; builds fail automatically on any unpatched CRITICAL or HIGH CVE.
- Cryptographic image signing — clean images are signed with Cosign (Sigstore), with private keys secured in a Hardware Security Module.
- Admission enforcement — the in-cluster admission controller verifies the Cosign signature and attestation provenance against the enterprise public key before scheduling; unsigned containers are rejected outright.

7.2 Runtime Threat Detection with Falco

Where admission control secures the pre-deployment phase, Falco inspects live system calls from the Linux kernel via an eBPF probe, catching intrusions that only manifest once a workload is running — for example, an attacker who has already achieved code execution attempting to spawn an interactive shell.

```

- rule: Unauthorized Shell Spawned in Production Container
  desc: Detects an interactive shell inside a financial microservice
  condition: >
    container.id != host and spawned_process and shell_procs and
    k8s.ns.name in (domain-payments, domain-risk, domain-customer)
  output: >
    CRITICAL: Shell spawned in protected financial container
    (user=%user.name pod=%k8s.pod.name ns=%k8s.ns.name
    image=%container.image.repository cmd=%proc.cmdline)
  priority: CRITICAL
  tags: [pci-dss, zero-trust, runtime-defense]
  
```

Listing 4. Falco rule detecting an unauthorised interactive shell in a protected financial namespace

8. Observability, Service Mesh & Scalability

8.1 Centralised Observability

The observability stack aggregates metrics, logs, and traces to satisfy PCI-DSS audit retention mandates. Prometheus scrapes system, CNI, and application-level metrics, exposing Rate/Errors/Duration (RED) indicators on Grafana dashboards. Cluster-wide log collectors stream API audit logs, container stdout, and Falco events to a centralised OpenSearch/Loki cluster, cryptographically signed and retained for an enforced 365-day immutable window. A Linkerd service mesh provides automatic mutual TLS across all inter-service traffic, with cryptographic identities rotated every 24 hours — encrypting east-west traffic without any change to application code.

8.2 Elastic Scalability & HPA Tuning

The Horizontal Pod Autoscaler control loop executes every 15 seconds, computing a target replica count from the standard Kubernetes autoscaling model:

$$R_target = \lceil R_current \times (M_current / M_desired) \rceil$$

where $R_current$ is the running replica count, $M_current$ is the current observed metric across all pods, and $M_desired$ is the threshold set in the HPA manifest. FinServ Digital's transaction-processor HPA is tuned for 65% average CPU utilisation and 1,200 m requests/second per pod, with an aggressive zero-second scale-up stabilisation window and a conservative five-minute scale-down cooldown to prevent flapping.

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: transaction-processor-hpa
  namespace: domain-payments
spec:
  minReplicas: 5
  maxReplicas: 25  # Accommodates 5x traffic surges
  metrics:
  - type: Resource
    resource: {name: cpu, target: {type: Utilization, averageUtilization: 65}}
  - type: Pods
    pods:
      metric: {name: http_requests_per_second}
      target: {type: AverageValue, averageValue: "1200m"}
  behavior:
    scaleUp: {stabilizationWindowSeconds: 0, policies: [{type: Percent, value: 100,
periodSeconds: 15}]}
    scaleDown: {stabilizationWindowSeconds: 300, policies: [{type: Percent, value: 10,
periodSeconds: 60}]}
```

Listing 5. HorizontalPodAutoscaler tuned for a 5x transaction burst with asymmetric scale-up/scale-down behaviour

9. PCI-DSS v4.0 Compliance Mapping Matrix

This matrix is the primary artefact presented to the Qualified Security Assessor (QSA) during the RBI-regulated PCI-DSS Level 1 audit. Each row traces a regulatory requirement through to the specific technical control that satisfies it and the evidence artefact that proves it.

Requirement	Regulatory Intent	Technical Control	Evidence Artefact
Req. 1	Install and maintain network security controls	Default-deny NetworkPolicies; domain namespace isolation; eBPF east-west microsegmentation	NetworkPolicy manifests; Calico verification logs
Req. 2	Apply secure configurations to all system components	Restricted Pod Security Standard; rootless execution; immutable filesystems; CIS benchmark hardening	Kyverno audit reports; kube-bench logs
Req. 3	Protect stored account data	Vault Transit field-level PAN encryption; etcd envelope encryption at rest (KMS)	Vault transit audit logs; KMS validation
Req. 4	Protect cardholder data in transit	Linkerd mTLS mesh; TLS 1.3 edge termination	Linkerd mTLS metrics; SSL Labs A+ verification
Req. 6	Develop and maintain secure systems and software	Distroless images; Trivy scanning; Cosign signing and admission control	CI/CD build traces; Sigstore signature logs
Req. 7	Restrict access by business need to know	Least-privilege RBAC; persona separation; OIDC-backed SSO	RoleBinding definitions; IAM audit logs
Req. 8	Identify and authenticate access to components	No default token automounting; ephemeral developer tokens; Vault identity engine	API server auth configs; Vault lease records
Req. 10	Log and monitor all access to system components	Centralised 1-year immutable logging; API audit logging; Falco runtime detection	Loki/OpenSearch retention policy; Falco dashboards
Req. 11	Regularly test security of systems and networks	Automated vulnerability scanning; runtime anomaly detection; isolated staging pen-tests	Trivy daily scan reports; Falco event history

10. Developer Onboarding & Self-Service Guide

This section equips application engineers with the exact procedures required to ship services safely inside the hardened platform.

Step 1 — Environment Authentication

```
# Authenticate via OIDC helper
kubectl oidc-login setup --oidc-issuer-url=https://auth.finservdigital.internal

# Scope your context strictly to your assigned domain namespace
kubectl config set-context finserv-dev --namespace=domain-payments
kubectl config use-context finserv-dev
```

Step 2 — Secrets Provisioning

Never store API keys, passwords, or connection strings in Git repositories or ConfigMaps. Register the application under its dynamic secrets path in the Vault portal, then reference it via Vault Agent annotations:

```
vault.hashicorp.com/agent-inject: "true"
vault.hashicorp.com/role: "<your-app-role>"
vault.hashicorp.com/agent-inject-secret-config.json: \
  "secret/data/payments/<your-service>"
```

Step 3 — Network Policy Authoring

Because every namespace defaults to deny-all, applications are network-isolated the moment they deploy. Identify every service your microservice calls (egress) and every service that calls yours (ingress), and include a network-policy.yaml inside your Helm chart declaring those paths down to port and namespace selector.

Step 4 — Pipeline Execution

- Linting — validates manifest syntax and confirms rootless security contexts.
- Scan — runs Trivy against the built image; CRITICAL CVEs fail the build immediately.
- Sign — the pipeline signs the verified image with Cosign.
- Deploy — the admission controller checks the signature and deploys to staging.

11. Enterprise Incident Response Playbook

The playbook below is triggered automatically the moment Falco raises a CRITICAL alert, and is designed to contain a compromised pod within thirty seconds without restarting the underlying host — preserving forensic state for later analysis.

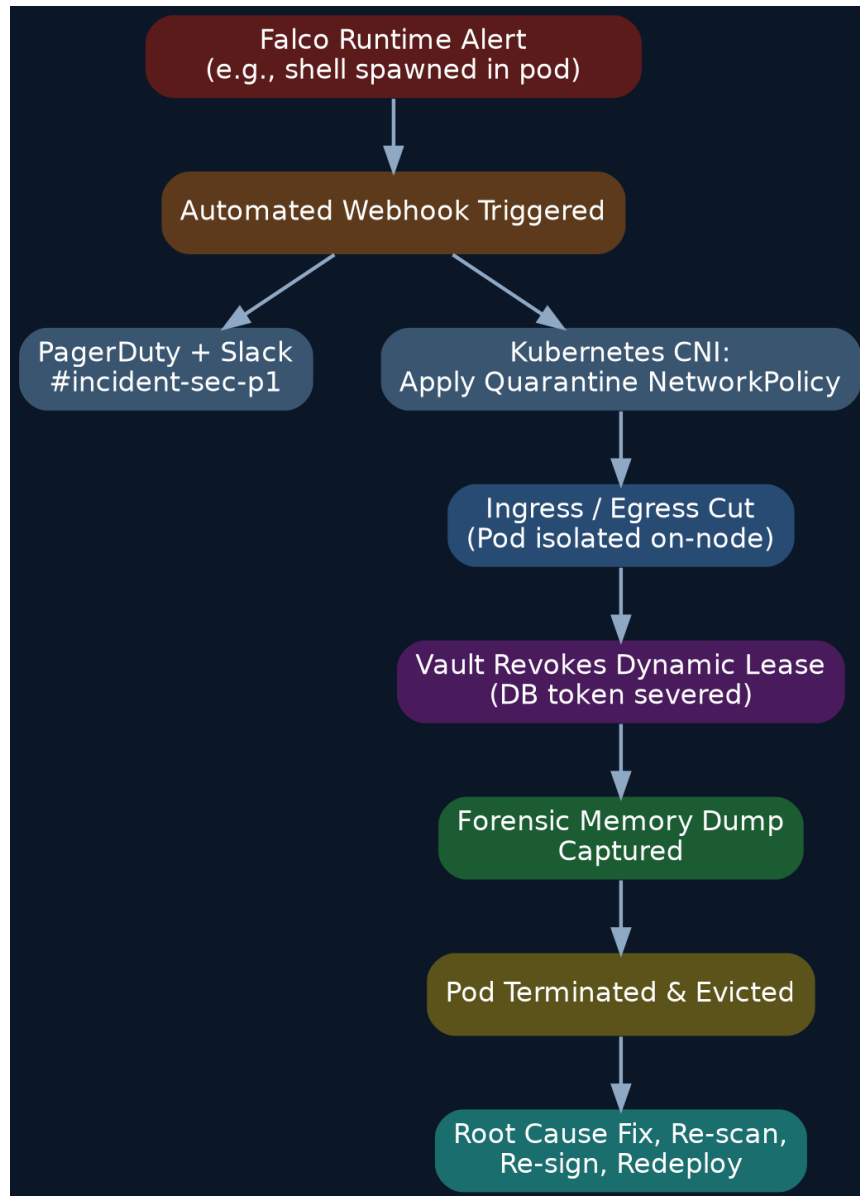


Figure 4. Automated incident response flow, from Falco detection through containment to root-cause remediation

Phase 1 — Detection & Triage

Falco raises a CRITICAL alert (for example, Unauthorized Shell Spawned in Production Container). The alert is auto-routed to the Security Operations Centre via PagerDuty and the #incident-sec-p1 Slack channel. The responder first inspects the container's execution context with a read-only query:

```
kubectl get pod <compromised-pod-name> -n domain-payments \
-o jsonpath='{.status.containerStatuses}'
```

Phase 2 — Automated Quarantine & Containment

```
# Apply the emergency quarantine label – severs ingress/egress
kubectl label pod <compromised-pod-name> -n domain-payments \
  security.quarantine=true --overwrite

# Revoke every Vault lease tied to the pod's service account
vault lease revoke -prefix database/creds/payments-db-role
```

Phase 3 — Forensic Extraction & Eradication

```
# Capture a memory snapshot via a dedicated debug agent
kubectl debug pod/<compromised-pod-name> -n domain-payments \
  --image=forensics-toolbox:latest --target=processor

# Force the ReplicaSet to spin up a clean, verified replica
kubectl delete pod <compromised-pod-name> -n domain-payments \
  --grace-period=0 --force
```

Phase 4 — Post-Mortem & Supply Chain Patching

The team reviews API-server audit logs to identify the initial entry vector, patches the vulnerable dependency, re-runs the Trivy/Cosign pipeline, and redeploys. An official incident post-mortem is drafted within 48 hours to satisfy RBI operational-risk reporting obligations.

12. 15-Day Implementation Methodology & Repository Structure

The migration is delivered across five sequential three-day phases, each ending with a concrete, verifiable artefact rather than a partial state.

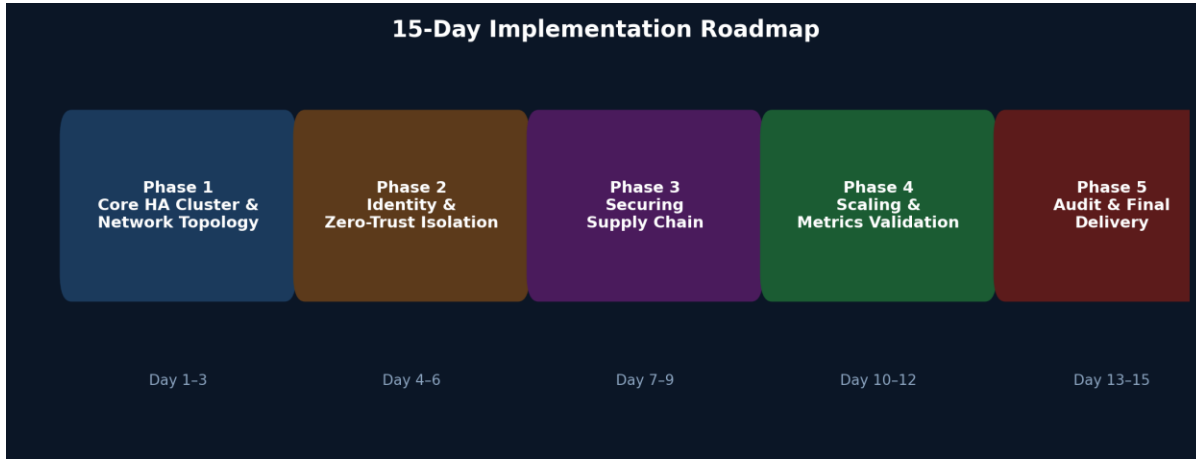


Figure 5. Fifteen-day phased delivery roadmap

Days	Phase	Key Deliverables
1–3	Core Cluster & Network Topology	Multi-AZ clusters provisioned; etcd HA with automated snapshots; 4 domain namespaces created; CNI deployed with default-deny baseline
4–6	Identity Governance & Secrets	OIDC integrated with API server; least-privilege RBAC applied; Vault HA deployed (Raft backend); Vault Agent Injector installed
7–9	Supply Chain & Runtime Defence	Distroless multi-stage builds; Trivy integrated into CI; Cosign keyless signing and Kyverno verification; Falco DaemonSet with PCI-DSS rules
10–12	Observability & Elasticity	Prometheus/Grafana/Loki with 1-year retention; Linkerd mTLS mesh; HPA tuned and tested against 5x surges; PodDisruptionBudgets set
13–15	Audit Simulation & Delivery	Red-team container-escape simulation; PCI-DSS control matrix finalised; onboarding & IR playbooks published; repository packaged for handover

12.1 Production Git Repository Structure

```

finserv-kubernetes-platform/
├── .github/workflows/
│   ├── ci-build-scan-sign.yaml    # Trivy scanning & Cosign signing
│   └── cd-deploy-staging.yaml     # Automated GitOps delivery
├── architecture/
│   ├── network-topology.png
│   ├── threat-model.md           # 4C security model docs
│   └── pci-dss-v4-mapping.md
├── manifests/
└── 00-namespaces/

```

```
|— 01-network-policies/
|— 02-rbac/
|— 03-vault/
|— 04-admission-control/
|— 05-runtime-security/
|— 06-observability/
|— 07-autoscaling/
|— docs/
|   |— developer-onboarding-guide.md
|   |— incident-response-playbook.md
|— README.md
```

13. Risk Analysis & Mitigation Register

Beyond the individual controls, it is worth naming the residual risks a review board would ask about directly, and how the architecture responds to each.

Risk	Likelihood	Impact	Mitigation
Vault becomes a single point of failure for all credential issuance	Low	High	Vault deployed in HA mode with Raft storage across AZs; short-TTL leases limit blast radius of any outage
Over-privileged break-glass access is misused	Low	High	Break-glass requires MFA, is time-boxed, and every action is captured in immutable audit logs
A supply-chain compromise inserts a malicious dependency	Medium	High	Trivy blocks CRITICAL/HIGH CVEs at build time; Cosign + admission control rejects any unsigned image
HPA scale-up lags a sudden transaction spike	Medium	Medium	Zero-second scale-up stabilisation window with 100%/15s scale-out policy; load-tested to 5x baseline
Alert fatigue causes a genuine Falco CRITICAL to be missed	Medium	High	Rules are scoped to protected namespaces only; CRITICAL routes to PagerDuty in addition to Slack

14. Future Work & Recommendations

- Extend Cosign attestation to include SLSA provenance metadata, moving from image signing toward full build-provenance verification.
- Introduce a policy-as-code review gate (Conftest/OPA) in pull requests, so NetworkPolicy and RBAC drift is caught before merge rather than at admission time.
- Pilot confidential computing (encrypted-memory nodes) for the domain-payments namespace to extend the trust boundary below the kernel.
- Automate quarterly chaos-engineering drills against the incident-response playbook, rather than relying solely on the Day 13–15 red-team simulation.
- Evaluate a second-region active-passive cluster for disaster recovery beyond the current multi-AZ, single-region high-availability model.

15. Conclusion

The architecture documented here replaces an unsegmented, statically-configured VM estate with a Kubernetes platform in which trust is never assumed and always verified — at the network layer through default-deny policy, at the identity layer through least-privilege RBAC, at the secrets layer through short-lived dynamic leases, at the supply-chain layer through signed and scanned images, and at runtime through continuous kernel-level detection. Each of these controls is traceable to a specific PCI-DSS v4.0 requirement, giving FinServ Digital both a defensible security posture and an audit-ready evidence trail. Delivered on a fifteen-day timeline, the platform is designed not as a one-time compliance exercise but as an operating model: every control in this dossier is enforced continuously, by policy, rather than by process discipline alone.

16. Appendix: Glossary of Acronyms

Acronym	Definition
AML	Anti-Money Laundering
AZ	Availability Zone
CNI	Container Network Interface
CVE	Common Vulnerabilities and Exposures
eBPF	extended Berkeley Packet Filter
HPA	Horizontal Pod Autoscaler
HSM	Hardware Security Module
IdP	Identity Provider
KMS	Key Management Service
KYC	Know Your Customer
mTLS	Mutual Transport Layer Security
OIDC	OpenID Connect
PAN	Primary Account Number
PCI-DSS	Payment Card Industry Data Security Standard
PSS	Pod Security Standards
QSA	Qualified Security Assessor
RBAC	Role-Based Access Control
RBI	Reserve Bank of India
SOC	Security Operations Center
TTL	Time To Live